

700106/700120 Final lab: Sandy-Snow Globe / Snowy-Sand Globe

Concept

The desert is a lonely place but teeming with life. The desert is caught up in a globe of very thin translucent material. Sand is everywhere on a plane inside the globe. The sand is swept by wind and everything is burned by the sun. Saguaros and Joshua trees are the only plants that thrive. The occasional desert mouse or even an armadillo is roaming around.

There is a day-night cycle as well as a seasonal cycle. By day the sun will shine and sometimes sets the plants on fire. By night temperatures will drop and the occasional snow cloud might drift through. There are only two seasons in the desert: dry-hot and dry-cold with rain occasionally mixed in.

Mandatory Features

Per-vertex shading (Gouraud shading)

Implement Gouraud shading in the vertex shader to perform per-vertex lighting. The technique should be applied to a high-polygon mesh, demonstrating effective illumination of densely tessellated geometric models.

Per-pixel shading (Phong shading)

Implement Phong shading in the pixel shader to perform per-fragment lighting. The technique should be applied to a low-polygon mesh, demonstrating accurate surface illumination through per-pixel normal and light vector evaluation.

Globe

The globe should have a diameter of 0.2 of a kilometre (or 200 units). It should be translucent from the outside but may show some environment-mapped sky effect if the viewer is inside the globe. The globe may be created from a mesh.

Ground Plane

The desert is a simple mesh (or tiles of meshes) with textures for adding the appearance of sand and is positioned as a plane at the equator of the globe. Terrain features, such as elevation changes or hills/valleys, can be created by pre-distorting the mesh's height values.

Plants

The desert plane should contain a couple of cacti. These do not have to resemble actual cacti but can be made rather abstract, i.e. some cylinders or capped cones for the main trunks and small-angled cones for the "leaves". The cacti should be texture-mapped. These plants react to the environment conditions: they start burning up if the sun shines for too long without rain, they also grow rapidly during and after rain or snow fall provided the sun is shining. As a guide, Saguaros can grow to as high as 20 metres.

Objects

You are required to place objects on the desert floor that demonstrates one or more of the shading, animation and lighting effects listed below.

The type of object are undefined, although they should be different, and selected to clearly enhance the chosen shading, lighting, animation effects.

At least one of the objects should consist of a mesh containing 1000's of polygons

Model loader

You are required to write your own model loader for the Wavefront OBJ file format [Wavefront OBJ File Format Summary](#).

Your model loader should be able to load basic models (i.e. those required for this assessment). There is no requirement to load all models supported by the OBJ format.

Day/Night/Seasonal Cycle

The sun is shining by day. By night a moon provides some lighting. Both should be implemented as directional light sources that change apparent direction over the day/night cycle. Over time seasonal changes take place. This can be modelled by signifying rain or snow fall.

Particle System(s)

There is ample opportunity for adding particle-system effects: fire, rain, snow, dust, sand storms, and probably many more. At least two particle-system effects should be implemented.

Shadows

All light sources should cast shadows. The shadows should use the shadow-mapping technique by default.

Cameras

- Camera C1 Overview camera at the outside of the globe positioned so that the globe (nearly) fills the viewport
- Camera C2 Navigation camera with initial position at an element that will exhibit (visible) shadow changes or a particle-system effect.
- Camera C3 Close-up camera directed at a cactus.

Controls

- 'ESC' exits the application
- 'r' resets the application to its initial state
- Cameras are controlled by the cursor keys:
 - 'left'/'right'/'up'/'down' rotate to the left/right/up/down, respectively
 - CTRL + 'left'/'right'/'up'/'down'/'page up'/'page down' panning to left/right/forward/backward/up/down, respectively
 - Alternatively, w-a-s-d or i-j-k-l keys may be used instead of the "cursor" keys
- Function keys F1 to F3 will select cameras C1 to C3, respectively

- Function key F4 starts a particle effect on demand (e.g., burning up a cactus), the camera needs to be adjusted so the viewer is able to see this!
- Keys 't'/'T' decrease/increase a factor that globally slows/speeds-up time-dependent effects

Configuration File

A configuration file should allow for positioning plants, animals, etc. in the scene.

Optional/Advanced Features

Each advanced feature has an associated difficulty point. The maximum mark available for each feature is this difficulty point. Only 20 marks are available for implementing advanced features. These 20 marks can be gained by either perfectly implementing features worth 20 points, or by partially implementing features worth up to 30 points. Any features implemented beyond 30 points will not be marked. Marks for advanced features will be capped at 20.

- **[4 points]** The globe can also be represented as a render pass whose behaviour with respect to appearance depends on the viewer position (inside vs. outside)
- **[2 points]** The configuration file can also be used to provide values for the initial direction of the sun/moon, the season lengths, or if/when environment effects happen.
- **[4 points]** Bump mapping to simulate elevated landscape on the inside of the globe
- **[2 points]** Displacement mapping to simulate elevated landscape on the inside of the globe
- **[4 points]** Environment mapping to be used for environment reflections on scene elements
- **[4 points]** Visualization of the sun and moon on the environment map for the inside of the globe with their apparent movement over time; if the viewer is outside the globe the sun/moon could be represented as (small) emissive spheres around the globe
- **[6 points]** Shadows
- **[4 points]** Fake turbulent dust cloud, wafting through the desert at random, but originating in the centre of the globe and moving in a random direction
- **[2 points]** Statistics display for graphics and simulation information, toggled with key 's'
- **[4 points]** Statistics and control elements using IMGUI; for the control elements this is in addition to the mandatory keyboard-navigation features, for the statistics this would replace the simple statistics above
- **[8 points]** Deferred rendering, including the ability to switch the visualization for the various MRT buffers with key 'd'
- **[4 points]** High-dynamic range rendering and tone-mapped post-processing
- **[4 points]** Illuminating sparks intermixed with the fire particle system; the sparks are to be implemented as omnidirectional short-ranged light sources, which illuminate near objects within the scene.

Novel features

- **[6 points]** Extra marks are awarded for any novel features that enhance the simulation, beyond the initial specification.

UI

Statistics and control elements using IMGUI

Hints

The following may help you to successfully complete your work.

1. Start now!
2. Produce a paper visualization (e.g., story board sketches) of your project and check it with the lecturers
3. Produce a top level software design
4. Prototype your ideas to help produce a more detailed design
5. Test your software at each stage of development
6. Document as you go
7. Make use of CoPilot
8. Adhere to the specification of features, especially the assignment of keyboard controls to features! If in doubt check it with the lecturers

Implementation

The final implementation must be able to run on a standard Windows 11 PC within RBB-335

Only the following libraries and APIs are permitted:

- Vulkan
- ImGui
- GLM
- GLFW
- STB Image

The use of any language feature within C++ 11, 14, 17, 20 or 22 is permitted (provided it is supported by the Microsoft compiler within VS2022).

Code submission

Please ensure the following requirements are met when submitting your source code.

- only submit source code that is used in this application (ie. not all your tutorials)
- clean the solution prior to submission
- ensure that the solution builds, without the need to install any additional libraries
- ensure that Parasoft C++ Static Test runs on the solution
- suppress Parasoft C++ Static Test warnings from any library header files, by placing these comment blocks at the very beginning and end of the files

```
// parasoft-begin-suppress ALL "suppress all violations"
```

```
// parasoft-end-suppress ALL "suppress all violations"
```

The code can ONLY be submitted via the provided GitHub Classroom repository (ie Lab I (the final lab), a link to which is provided on Canvas)

It is your responsibility to make very sure that all the source code files required to make your application are committed to the GitHub Classroom repository. It's easy to forget to add or push some while you are developing. A good way to check this is to check out your project to a different folder and see if you can build it as follows:

1. Clone your repository in to an empty folder.
2. Open the solution in Visual Studio
3. Run your project(s). Remember that to achieve full marks your submitted code must compile and run.
4. If any of those previous steps doesn't work you probably forgot to add/commit/push one or more of your files. GOTO step 1.

Do not leave it until the last minute to commit and push your work. Committing your repository all in one go may take upwards of an hour to complete. When you do commit your work it is recommended that you include the phrase "ACW FINAL" in the commit log. This indicates that you have made a submission. You can do this multiple times before the deadline if you need to. Submissions made after the deadline will be subject to the usual penalties.

Lab Book

You are required to add the following two entries to your Lab Book:

Design (700120) - [word limit 1000]

- A textual description of each class giving the name, role and responsibilities
- Mermaid UML Class diagram(s) containing
 - main classes
 - service/utility classes
- Mermaid UML Interaction diagram(s) for significant components of the software design
- A critique of the design, including
 - The merits of the design
 - Weaknesses of the design
 - What would you change and why?

Graphics (700106) - [word limit 1000]

- Document and critique the algorithms used, including:
 - Geometry representation and processing
 - Shading and lighting
 - Shadow generation
- Explain how application objects and their graphics representation are connected; discuss the advantages/disadvantages of your approach
- Explain how application-object behaviour is updated and how these updates are propagated to the graphics representation
- Discuss potential extensions (especially with respect to non-implemented items from the advanced feature list) as well as potential scalability issues
- What feature (not necessarily mentioned before) would you have liked to add?

The lab book is to be submitted alongside your code on GitHub

Please note, that your report should adhere to the document standards in Computer Science. Marks will be lost if you exceed word limits; see handbook for over length penalties!

Video

A 3 to 5 minute video with audio commentary. The video should highlight each of the key elements of your implementation. It is strongly suggested that you work through the list of mandatory and optional features within the specification, showing each in turn. The video is to be submitted via Canvas

Demonstration

You will be required to demonstrate your solution, the purpose of which is

- To assess the quality and quantity of the features you have chosen to implement. We'll work through each feature in the specification.
- To assess your knowledge and understanding of your implementation. We'll ask you a series of questions related to how and why you chose to implement a certain feature in the way you did.

We recognise that your solution may consist of a fairly large code base, some of which you may not have touched for several weeks prior to the demo. To simplify the process, we'll release a set of possible questions before the demonstration, to enable you to prepare your answer.